Drone Detection Model Comparison App Deployment Guide

VisionSentry Project

April 3, 2026

Contents

1 Introduction

This guide provides step-by-step instructions for deploying the Drone Detection Model Comparison web application. The application uses YOLOv11/12 models for drone detection and provides a web interface for comparing two models side-by-side.

1.1 Key Features

- Side-by-side model comparison
- Synchronized video playback
- Real-time detection visualization
- Detection statistics and graphs
- CPU-optimized for cloud deployment

1.2 System Requirements

- Python 3.11+
- 2GB+ RAM (4GB recommended)
- CPU-only (no GPU required)
- Docker (for containerized deployment)

2 Deployment Options

2.1 Why Not Vercel?

Important Note

Vercel does not support Python applications. Vercel is designed for JavaScript/Node.js frameworks (Next.js, React, etc.) and static sites only. For Python ML applications, use the alternatives below.

2.2 Recommended Platforms

1. **Hugging Face Spaces** - Free, ML-optimized, easiest deployment
2. **Railway.app** - Simple, free tier available
3. **Render.com** - Free tier, Docker support
4. **Google Cloud Run** - Pay-per-use, scalable
5. **AWS EC2** - Full control, requires more setup

3 Option 1: Hugging Face Spaces (Recommended)

3.1 Why Hugging Face Spaces?

- **Free tier** with generous resources
- Optimized for ML applications
- Built-in Gradio support
- No credit card required
- Automatic HTTPS

3.2 Deployment Steps

3.2.1 Step 1: Create Account

1. Go to `https://huggingface.co`
2. Sign up for a free account
3. Verify your email

3.2.2 Step 2: Create New Space

1. Click on your profile → "Spaces"
2. Click "Create new Space"
3. Configure:
    - Space name: `drone-detection-comparison`
    - License: MIT or Apache 2.0
    - SDK: **Gradio**
    - Hardware: CPU basic (free)
4. Click "Create Space"

3.2.3 Step 3: Upload Files

Upload the following files to your Space:

```
model_comparison_app.py
requirements_app.txt
weights/best.pt
VisionSentry-dex-rgb-model-replication/weights/best.pt
README.md
```

3.2.4 Step 4: Configure App File

Rename `model_comparison_app.py` to `app.py` (Hugging Face convention):

```
mv model_comparison_app.py app.py
```

3.2.5 Step 5: Deploy

1. Commit files to the Space repository
2. Hugging Face will automatically build and deploy
3. Wait 2-5 minutes for deployment
4. Access your app at: `https://huggingface.co/spaces/YOUR_USERNAME/drone-detection-comparison`

4 Option 2: Railway.app

4.1 Deployment Steps

4.1.1 Step 1: Setup

1. Go to `https://railway.app`
2. Sign up with GitHub
3. Click "New Project" → "Deploy from GitHub repo"

4.1.2 Step 2: Configure

1. Select your repository
2. Railway will detect the Dockerfile
3. Set environment variables (if needed)
4. Click "Deploy"

4.1.3 Step 3: Access

1. Wait for build to complete (5-10 minutes)
2. Railway will provide a public URL
3. Access your app at the provided URL

5 Option 3: Render.com

5.1 Deployment Steps

5.1.1 Step 1: Create Account

1. Go to `https://render.com`
2. Sign up with GitHub

5.1.2 Step 2: Create Web Service

1. Click "New +" → "Web Service"
2. Connect your GitHub repository
3. Configure:
   - Name: `drone-detection`
   - Environment: Docker
   - Plan: Free

5.1.3 Step 3: Deploy

1. Render will use the `render.yaml` configuration
2. Click "Create Web Service"
3. Wait for deployment (10-15 minutes first time)

6 Local Deployment

6.1 Using Docker

6.1.1 Build Image

```
cd VisionSentry
docker build -t drone-detection-app .
```

6.1.2 Run Container

```
docker run -p 7860:7860 drone-detection-app
```

6.1.3 Access

Open browser to: `http://localhost:7860`

6.2 Using Python Directly

6.2.1 Install Dependencies

```
pip install -r requirements_app.txt
```

6.2.2 Run Application

```
python model_comparison_app.py
```

6.2.3 Access

Open browser to: `http://localhost:7860`

7 Configuration

7.1 CPU vs GPU

The application is configured to use **CPU by default** for deployment compatibility:

```
# CPU inference (default)
results = model(image, device='cpu')
```

To enable GPU locally (if available):

```
# GPU inference
results = model(image, device='cuda')
```

7.2 Model Paths

Update model paths in `model_comparison_app.py`:

```
model1_path = "weights/best.pt"
model2_path = "VisionSentry-dex-rgb-model-replication/weights/best.pt"
```

7.3 Port Configuration

Default port is 7860. To change:

```
demo.launch(server_port=8080)
```

8 Troubleshooting

8.1 Common Issues

8.1.1 Out of Memory

Solution: Reduce video resolution or process fewer frames at once.

8.1.2 Slow Inference

Solution: CPU inference is slower. Consider upgrading to a paid tier with GPU support on Hugging Face Spaces.

8.1.3 Model Not Found

Solution: Ensure model weights are uploaded and paths are correct.

8.2 Platform-Specific Issues

8.2.1 Hugging Face Spaces

- **Build fails:** Check `requirements_app.txt` for incompatible versions
- **App crashes:** Check logs in Space settings
- **Slow loading:** Upgrade to better hardware tier

8.2.2 Railway/Render

- **Deployment timeout:** Large model files may cause timeout. Use Git LFS
- **Port binding:** Ensure app binds to `0.0.0.0`, not `localhost`

9 Performance Optimization

9.1 Reduce Model Size

- Use smaller YOLO models (YOLOv11n instead of YOLOv11x)
- Quantize models for faster inference
- Use ONNX format for optimized inference

9.2 Video Processing

- Limit video length (e.g., max 30 seconds)
- Reduce frame rate (process every Nth frame)
- Lower output resolution

9.3 Caching

- Cache model loading
- Use Gradio's built-in caching
- Implement result caching for repeated inputs

10 Security Considerations

10.1 Best Practices

1. **Input validation:** Limit file sizes and formats
2. **Rate limiting:** Prevent abuse with request limits
3. **HTTPS:** All platforms provide automatic HTTPS
4. **Environment variables:** Store sensitive data securely

10.2 File Upload Limits

```
# Limit video size to 100MB
video_input = gr.Video(label="Upload Video",
                       file_count="single",
                       max_size=100*1024*1024)
```

11 Monitoring and Maintenance

11.1 Logging

Enable logging for debugging:

```
import logging
logging.basicConfig(level=logging.INFO)
```

11.2 Usage Analytics

- Hugging Face Spaces provides built-in analytics
- Railway/Render offer usage dashboards
- Monitor CPU/memory usage

11.3 Updates

1. Test changes locally first
2. Use version control (Git)
3. Deploy to staging before production
4. Monitor after deployment

12 Cost Estimation

12.1 Free Tiers

- **Hugging Face Spaces:** Free CPU tier (2 vCPU, 16GB RAM)
- **Railway:** $5 free credit/month
- **Render:** 750 hours/month free

12.2 Paid Tiers (if needed)

- **Hugging Face GPU:** $0.60/hour (T4 GPU)
- **Railway:** $0.000231/GB-second
- **Render:** $7/month (starter)

13 Quick Start Checklist

☐ Choose deployment platform (Hugging Face recommended)

☐ Create account on chosen platform

☐ Prepare files: `app.py`, `requirements_app.txt`, model weights

☐ Upload files to platform

☐ Configure settings (CPU, port, etc.)

☐ Deploy and wait for build

☐ Test application with sample video

☐ Share public URL

14 Support and Resources

14.1 Documentation

- Gradio: `https://gradio.app/docs`
- Hugging Face Spaces: `https://huggingface.co/docs/hub/spaces`
- Ultralytics YOLO: `https://docs.ultralytics.com`

14.2 Community

- Gradio Discord: `https://discord.gg/gradio`
- Hugging Face Forums: `https://discuss.huggingface.co`

15 Conclusion

This guide covered multiple deployment options for the Drone Detection Model Comparison app. For most users, **Hugging Face Spaces** is the recommended platform due to its ease of use, ML optimization, and generous free tier.

For production deployments with high traffic, consider upgrading to paid tiers or using cloud platforms like Google Cloud Run or AWS with auto-scaling capabilities.

Happy Deploying!